Computer Vision: Exam Question Bank (57 questions), answered from your course files only

Every answer below uses **only** your course material: the lecture PDFs, Key Concepts PDFs, lab notebooks and GAN slides/guides. Nothing from outside the course is added. Each answer ends with its **Source**. Where your files don't cover something a question asks, the answer says so with ⚠, instead of filling the gap from outside.

Source key - **W1-KC**: `week-1/course-files/lecture-pdfs/Key_Concepts.pdf` - **D1-3 / D4 / D5 / D6**: the Week 1 lecture PDFs (`Days_1-3_Lectures.pdf`, `Day_4_Histograms_and_Thresholding.pdf`, `Day_5_Morphological_Operations.pdf`, `Day_6_Contours_and_Shape_Analysis.pdf`) - **W2-KC**: `week-2/course-files/CV-Week-2-KeyConcepts.pdf` · **W2-Lab n**: `week-2/course-files/labs/CV-Week2-Lecture{n}.ipynb` - **W3-KC**: `week-2-3/course-files/CV-Week3-KeyConcepts.pdf` - **GAN-1 / GAN-2-3**: `week-4-gans/course-files/GAN_Class_1_Basics.pptx` / `GAN_Class_2_3_Training_Dynamics.pptx`

Short answers (1–19)

1. Define a pixel in the context of digital images. Digital images are **grids of pixels**. An image is stored as a matrix of numbers, where **each number is the intensity or colour value of one pixel**. In a grayscale image a pixel is a single number (0 = black, 255 = white). In a colour image each pixel has 3 values (R, G, B; OpenCV stores them as B, G, R). The origin (0, 0) is the top-left corner, and a pixel is accessed as `image[y, x]`. *Source: W1-KC (Image Representation Basics); D1-3 (Images as Data).*

2. What is the difference between grayscale and color image representation? - **Grayscale → 2D matrix:** one channel, and each pixel is a single number from 0 (black) to 255 (white). `img_gray.shape` = (rows, cols). - **Colour (RGB/BGR) → 3D matrix:** three channels, and each pixel has 3 values, e.g. [255, 0, 0] = pure red. `img_color.shape` = (rows, cols, 3).

Source: W1-KC; W2-KC (Image Basics: "Grayscale = one channel, RGB = three channels").

3. Why is OpenCV's color order BGR instead of RGB? Your course states the fact and its consequence: **OpenCV uses BGR order (Blue, Green, Red), while Matplotlib expects RGB.** So before displaying an OpenCV image with Matplotlib you convert it with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`, as every course notebook does. Printing a pixel, e.g. `img[50, 50]`, gives **BGR** values. ⚠ *Your course files do not explain **why** OpenCV chose BGR; they only state that it does. Source: W1-KC; D1-3.*

4. What does the `img.shape` attribute return in NumPy? The array's dimensions: - **colour image:** `(height, width, channels)` (rows, cols, 3), e.g. `(5472, 3648, 3)` in the Day 1 lecture; - **grayscale image:** `(rows, cols)`.

`rows, cols = img.shape[:2]` is used before transformations. *Source: D1-3 ("Image shape: (height, width, channels)"); W1-KC.*

5. Define a feature in computer vision. Features are **distinctive, informative patterns in an image that can be detected, described, and compared**. They are the building blocks for recognition, matching and tracking. Kinds: - **corners:** intensity changes in two directions; - **edges:** boundaries between regions of different brightness or texture; - **blobs/keypoints:** regions that stand out from their surroundings; - **descriptors:** numerical vectors describing the neighbourhood.

Good features have **uniqueness**, **repeatability** and give a **compact representation**. *Source: W2-KC (What Features Represent?).*

6. What is a keypoint? A keypoint is a **specific location (x, y) in an image that the algorithm identifies as distinctive and informative**: the "coordinates" of a feature such as a corner, blob or edge. In SIFT and ORB each keypoint stores **position (x, y), scale, orientation and response** (strength of detection). Keypoints serve as **anchors** for describing and matching images. *Source: W2-KC (What are Keypoints?).*

7. State one advantage of ORB over SIFT. ORB is **free, fast and efficient**, and suitable for real-time tasks. SIFT is **computationally heavy** and was historically patented (now free in OpenCV ≥ 4.4). ORB is also rotation invariant. *Source: W2-KC; W2-Lab 1.*

8. What does pooling achieve in CNNs? Pooling **down-samples feature maps by summarising regions** (max pooling keeps the strongest feature; average pooling smooths). It: - reduces computation and dimensionality; - provides **translation invariance** (robust to small shifts); - controls overfitting.

Source: W2-KC (Pooling; What Pooling Does?).

9. Define ReLU and explain its purpose. ReLU (Rectified Linear Unit): if the input is positive, keep it; if negative, output zero. Negative values are "cut off" and positives pass through. Purpose: - it introduces **non-linearity** so networks can learn complex patterns; - it is **faster and simpler** than sigmoid or tanh; - it **helps reduce the vanishing gradient problem**.

It was one of AlexNet's innovations (faster training). *Source: W3-KC (ReLU and Dropout).*

10. What is dropout and why is it used? Dropout is a **regularisation technique to prevent overfitting**. It **randomly "drops" (sets to zero) a fraction of neurons during training**, which forces the network to learn robust features instead of memorising, reduces reliance on specific neurons, and **improves generalisation**. *Example:* with 10 neurons and dropout rate 0.3, about 3 neurons are ignored in each training pass. *Source: W3-KC.*

11. State one key contribution of LeNet. LeNet (1998, **Yann LeCun**, digit recognition on MNIST) was the **first proof that CNNs outperform traditional ML methods (like SVMs) for images**. It inspired early adoption in handwriting recognition and banking systems. Architecture: Convolution → Pooling → Fully Connected. *Source: W3-KC.*

12. What does "compound scaling" mean in EfficientNet? Compound scaling **scales depth, width and resolution systematically using a single formula** (balanced scaling across dimensions), instead of scaling them unevenly. It achieves high accuracy with fewer parameters. *Source: W3-KC (EfficientNet).*

13. Define viewpoint variation in computer vision. ⚠ *"Viewpoint variation" is not defined as a term in your course files.* The closest material: - Day 2 lists **perspective transformation** as "warping viewpoint"; - stereo vision uses "two images from slightly different viewpoints"; - features must be **repeatable**: "the same feature should be detectable even if the image changes (scale, rotation, lighting)"; - Haar cascades work for **frontal** faces and are sensitive to **pose**.

From these: viewpoint variation is when the same object looks different because it is seen from a different angle/perspective, so detectors need features that stay repeatable under such changes. *Source: D1-3; W1-KC; W2-KC.*

14. What is occlusion, and why is it challenging? Occlusion means the **object is hidden**: the KLT tracker "struggles with occlusion (object hidden)". It's challenging because the hidden object can't be seen. The course lists it as a challenge for **object tracking** ("occlusion, scale changes, multiple objects, cluttered backgrounds"), for **optical flow** ("sensitive to lighting changes and occlusions"), and for **Haar face detection** ("sensitive to lighting, pose, and occlusion"). *Source: W2-KC; W1-KC; W2-Lab 3.*

15. Define a GAN in simple terms. GAN = Generative Adversarial Network: - **Generative:** it creates new data (new images, not labels); - **Adversarial:** it learns through a contest between two opponents; - **Network(s):** both players are neural networks trained with backpropagation.

A **Generator** turns random noise into fake images, and a **Discriminator** judges real vs fake. Each one's improvement forces the other to improve. *Source: GAN-1 (slides 6, 12–14).*

16. What dataset is used in the notebook? MNIST (Modified National Institute of Standards and Technology): handwritten digits 0–9. It has 70,000 images (60,000 train + 10,000 test), each **28 × 28 = 784 pixels**, greyscale 0–255, 10 classes. The notebook loads `datasets.MNIST(root='./data', train=True, ...)` with batch size 64. *Source: GAN-1 (slides 46, 49).*

17. What is the input to the Generator? A **random noise vector z of 100 numbers**, `torch.randn(batch_size, 100)`: "a random vector, not a photo, not a label". The space z lives in is the **latent space**. *Source: GAN-1 (slides 21, 22, 25).*

18. What does the Discriminator output? One number: P(real), a probability between 0 and 1 produced by a **Sigmoid**. Near 1 = "I'm fairly sure this is real"; near 0 = "fake". *Source: GAN-1 (slides 27, 30).*

19. Why do we usually keep only the Generator after training? The Generator **makes the images**, while D is "usually discarded". D's job was to **push G to improve** during training: "opponents during training, but partners in the result". After training, **only G is needed**: new noise → brand-new digits. *Source: GAN-1 (slides 16, 31, 50).*

Medium answers (20–38)

20. Explain the role of Hue, Saturation, and Value in color representation. HSV is a way of representing colours **closer to how humans perceive them** than RGB: - **Hue (H):** the **type of colour** (red, green, blue…), measured as an **angle on a colour wheel** (0° = red, 120° = green, 240° = blue). - **Saturation (S): how intense or pure** the colour is: 0% = completely grey, 100% = fully vivid. - **Value (V):** the **brightness**: 0% = black, 100% = the brightest version of the hue.

Separating colour into these intuitive parts makes HSV useful for **colour detection, segmentation and image editing**. E.g. ripe fruit can be detected by isolating the hue range for "yellow" or "red" **without worrying about brightness changes**. Conversion: `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`. *Source: W1-KC (HSV); D1-3.*

21. What is the purpose of a kernel in image processing? A kernel (also called a filter, mask or convolution matrix) is a **small matrix (3×3, 5×5, 7×7) that slides over an image to modify it**: to blur, sharpen or detect edges. It **determines how pixel values are weighted and combined during convolution**: 1. place the kernel over a pixel and its neighbours; 2. multiply each kernel value by the pixel under it; 3. sum the results → the new pixel value.

Examples: - identity [0 0 0; 0 1 0; 0 0 0] leaves the image unchanged; - box blur (1/9 of all ones) smooths; - sharpen [0 −1 0; −1 5 −1; 0 −1 0] enhances edges and fine details; - Sobel X [−1 0 1; −2 0 2; −1 0 1] finds vertical edges, and Sobel Y finds horizontal edges.

Larger kernels capture more context but are slower. Kernels matter for **feature extraction, image enhancement**, and in **CNNs they are learned automatically**. *Source: W1-KC (What is a Kernel?).*

22. How does histogram equalization enhance image contrast? Your course covers this only at the level of histograms: - A histogram is a graph of **how many pixels have each intensity value** (x-axis 0–255, y-axis count). - It is used to **detect brightness and contrast levels** and to identify whether an image is **underexposed (too dark) or overexposed (too bright)**. - It is the "**basis for enhancement techniques like histogram equalization**".

So histogram equalization is an enhancement technique that works on the image's intensity histogram to improve contrast. ⚠ *Your course files name histogram equalization but do not explain how it works (no steps or formula). Source: W1-KC (Histograms); D4.*

23. Explain the concept of descriptors with examples. A descriptor **encodes the local neighbourhood around a detected keypoint into a numerical vector**. Two keypoints are a **match if their descriptors are similar**, so descriptors let features be compared across images. Detection finds the keypoints; description produces the descriptors. **Examples:** - **SIFT:** a 128-dimensional vector, robust to scale and rotation; - **ORB:** a binary string; fast, efficient, free; - **SURF:** 64 or 128 dimensions; faster than SIFT, but patented.

They're used in logo detection, object recognition and image retrieval. *Source: W2-KC (Feature Descriptors); W1-KC (SIFT, SURF); W2-Lab 2.*

24. How does FLANN improve feature matching efficiency? FLANN (Fast Library for Approximate Nearest Neighbours) is a library **optimised for fast matching of high-dimensional feature descriptors**. - **Pros:** efficient for large datasets; scales well with many features. - **Cons:** approximate, so it may miss exact matches; needs parameter tuning. - It is combined with **Lowe's ratio test** to filter weak or ambiguous matches. In the lab: `matches = flann.knnMatch(des1, des2, k=2)`, keeping a match only if `m.distance < 0.7*n.distance`. - It is used in the **Logo Detector** mini-project: detect features → match with FLANN → ratio test → homography → bounding box.

Source: W2-KC (FLANN); W2-Lab 2.

25. What is the role of RANSAC in robust estimation? RANSAC is a **robust estimation method to handle outliers in matches**. When estimating a **homography** (the transformation mapping one image plane to another), some feature matches are wrong. RANSAC makes the estimate robust to them. The course uses it in: - **panorama stitching:** detect ORB features → match → estimate homography → warp and stitch; - the **logo detector:** FLANN matches → homography → box around the logo.

Code: `M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)`. ⚠ *Your course files do not describe RANSAC's internal steps; they only give its role and the OpenCV call. Source: W2-KC; W2-Lab 1, 2.*

26. Explain the concept of skip connections in ResNet. ResNet (2015) solved the problem that training very deep networks led to **vanishing gradients** (weights stopped updating). It introduced **residual (skip) connections**, which bypass layers and feed the input to a later layer: - the network **learns residuals (differences) instead of direct mappings**; - skip connections **let gradients flow more easily** through the layers.

Impact: networks of **50, 101, even 152 layers** became trainable, with state-of-the-art ImageNet performance. Application: medical imaging. *Source: W3-KC (ResNet; skip connections).*

27. How does YOLO differ from R-CNN in object detection?

	R-CNN (2014)	YOLO (2016)
process	1. generate region proposals; 2. run a CNN on **each** region; 3. classify and refine boxes	treats detection as **a single regression problem**: boxes + class probabilities in **one forward pass**
strength	**high accuracy**	**real-time** performance
weakness	**very slow**: thousands of regions per image	less accurate for **small objects**
contribution	first major CNN-based detector	made detection practical for real time (self-driving cars)
best for	medical imaging (accuracy > speed)	autonomous driving (speed > accuracy)

Source: W3-KC (Object Detection Architectures).

28. What is the role of the encoder and decoder in U-Net? - **Encoder (downsampling path):** gradually reduces spatial resolution using **convolution + pooling**, capturing **context and high-level features**: it learns **what** is in the image ("this region looks like a tumour"). Analogy: compressing a photo into a summary. - **Decoder (upsampling path):** gradually increases resolution back to the original size using **transposed convolutions or upsampling**, reconstructing **pixel-level predictions**: it learns **where** things are. - **Skip connections** bridge the two: they combine low-level fine detail with high-level context, giving precise boundaries.

"Encoder compresses for context, decoder reconstructs for precision." U-Net is the standard in medical image segmentation. *Source: W3-KC (U-Net; Encoder; Decoder).*

29. Explain how histogram equalization helps with illumination changes. What your course provides: - **Histograms** reveal brightness and contrast problems (under- or over-exposure), and are the basis for **histogram equalization** as an enhancement technique. - For **varying illumination**, the course's tool is **adaptive thresholding**: "threshold value is calculated for smaller regions; useful for images with varying illumination", as opposed to one global cutoff. - Among features, **HOG** is "robust to illumination changes", and **SIFT** "handles changes in scale, rotation, and illumination".

⚠ *Your course files do not explain how histogram equalization handles illumination changes; only the items above are covered. Source: W1-KC; D4; W2-KC.*

30. Explain why normalization of MNIST images is important for GAN training. The notebook transforms each image with: 1. `ToTensor()`: pixels 0–255 → **0.0–1.0**; 2. `Normalize((0.5,), (0.5,))`: (v − 0.5)/0.5 → **−1 to +1**. E.g. pixel 204 → 0.8 → +0.6; pixel 0 → −1.0.

This matches the range of the **Generator's Tanh output (−1…1)**: "Tanh squashes outputs to −1…1, the same range as the real images after Normalize". Real and fake images therefore live in the same value range. *Source: GAN-1 (slides 25, 47, 49).*

31. Describe the two steps in each training round (update D, update G). Every mini-batch plays one round in two steps: - **Step 1: train the inspector (D); G is frozen.** - Real batch → D → BCE loss with **target 1** (`labels_real`): penalise D if it calls real images fake. - Noise → G → fakes → D with **`fake.detach()`** → **target 0** (`labels_fake`): penalise D if it calls fakes real. - `loss_d = loss_real + loss_fake`, then `loss_d.backward(); optimizer_d.step()`. - `.detach()` cuts the wire back to G and only `optimizer_d` steps, so this step changes **only D's weights**. - **Step 2: train the forger (G); D is not updated.** - The same fakes **without detach** → D → BCE with **target 1** (`labels_real`): "loss is low when D is fooled". - Gradients flow **through D back into G**, and only `optimizer_g` steps.

Source: GAN-1 (slides 33–35).

32. What is the role of random noise vectors in generating new images? The noise vector z is **the source of variety**: "Different z, different flower". **G is deterministic, and all the randomness comes from z.** Sampling z is easy (`torch.randn(64, 100)`), and G learns to map this simple cloud of points onto the complicated set of real images. The space z lives in is the **latent space**. - Re-running the plotting with new noise gives **different digits without retraining**. - Keeping one fixed noise vector (`fixed_noise`) and plotting `G(z_fixed)` each epoch is "the best way to watch a GAN learn". - **Walking from z1 to z2** changes the digit smoothly (Task 4).

Source: GAN-1 (slides 22, 23, 51, 53, 54).

33. Describe the mathematical operation of convolution and its role in feature extraction. Convolution is **a mathematical operation where a small matrix (kernel) slides over the image, multiplying and summing pixel values to produce a new output**:

$(I * K)(x, y) = \Sigma_m \Sigma_n I(x - m, y - n) \cdot K(m, n)$, where I is the image and K the kernel.

- The sliding window computes a **weighted sum**, producing a **feature map** that highlights patterns. Convolution is a **"sliding dot product"**.
- **Role in feature extraction:** it extracts edges, textures and smoothness (blur kernel → smoothing; edge kernel → boundaries; sharpen → fine detail).
- In **CNNs**, the filters are learned: **early layers → edges, corners, textures; deeper layers → shapes, objects, semantic features**. Convolutions **preserve spatial locality** and **reduce parameters** compared with fully connected layers.
- It is "the foundation of CNNs".

Source: W1-KC (Convolution); D1-3; W2-KC (Convolutions; Feature Maps; Mathematics Foundations).

34. Describe the difference between supervised and unsupervised learning with CV examples.

	Supervised	Unsupervised
data	**labelled** (inputs + correct outputs)	**unlabelled** (only inputs)
goal	predict outcomes for new data	discover hidden patterns, groupings, structure
techniques	classification, regression; kNN ("majority vote of neighbours"), SVM ("find the best dividing line")	clustering (k-means, hierarchical), dimensionality reduction (PCA, t-SNE), association
output	a class label or numeric value	groups, clusters, reduced representation
analogy	a teacher giving questions with answers	exploring a new city without a guide

CV examples: - **supervised:** handwritten digit recognition (MNIST) with kNN/SVM/CNN; object classification in photos; medical image analysis; - **unsupervised: organising large image collections**, and **clustering descriptors with k-means** into visual words (Bag of Visual Words).

Source: W2-KC (Machine Learning Fundamentals; BoVW).

35. Describe how Vision Transformers use attention to capture global context. CNNs excel at spatial features but **struggle with long-range dependencies**. ViT applies the transformer's **attention mechanism** to images: - **Attention:** Query (what we're looking for), Key (what we have), Value (the information to retrieve). Compare Query with Keys → similarity scores → use the scores to weight the Values → the output is a **weighted sum**. This captures **long-range dependencies** (pixels far apart but related). - **Step 1:** split the image into **patches** (e.g. 16×16); each is flattened and embedded as a **token**, like a word in NLP. - **Step 2:** a **transformer encoder** processes the patch sequence with attention, learning **relationships across patches (global context)**. - **Step 3:** a special **classification token** summarises the image for the final class.

Why patches give global context: - attention can directly compare far-apart regions (top-left vs bottom-right; zebra stripes in different parts of the body); - patches reduce the sequence length; - each patch keeps local detail, while attention connects patches globally.

Source: W3-KC (Vision Transformers & Attention).

36. Discuss how multi-scale feature extraction addresses scale variation in CNNs. From your course: - **SSD** "uses **multiple feature maps at different scales**" and so "detects small and large objects simultaneously" (e.g. a small cat and a large cat in the same image). This is why it balances speed and accuracy across scales, in contrast to **YOLO**, which is less accurate for small objects. - **CNN feature hierarchies:** early layers capture edges and textures, deeper layers shapes and objects. - **Classical methods: SIFT** builds a **Gaussian scale space** and detects Difference-of-Gaussians extrema, so it is robust to scale. **Harris** is not invariant to scale, ORB is less robust for large scale changes, and HOG is sensitive to scale changes. Haar face detection scans multiple scales with `detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)`.

Source: W3-KC (SSD); W2-KC; W1-KC (SIFT, HOG); W2-Lab 6.

37. Explain the minimax objective function of GANs.

min_G max_D V(D, G) = E_x[log D(x)] + E_z[log(1 − D(G(z)))]

- **Term 1 (real images):** the average of log D(x), largest (0) when D(x) = 1. **D maximises** it by saying "real" to real data. **G can't touch it.**
- **Term 2 (fakes):** D maximises it by pushing **D(G(z)) → 0**; **G minimises** it by pushing **D(G(z)) → 1**.
- **Minimax** = G minimises what the best D can maximise. It's a **zero-sum game**: D's gain is G's loss. Maximising V = minimising the notebook's `loss_d`.
- **Where it settles:** the best D for a fixed G is **D*(x) = p_data(x) / (p_data(x) + p_g(x))**; the best G gives **p_g = p_data → D*(x) = ½**, a Nash equilibrium. Reference: loss_d = 2 ln 2 ≈ 1.386 and loss_g = ln 2 ≈ 0.693.

Source: GAN-1 (slides 38, 39); GAN-2-3.

38. Discuss the three common problems in GAN training: mode collapse, instability, and D too strong. 1. **Mode collapse:** G finds one output that fools D and produces only that: every z gives the same kind of output. It looks good but has no variety (e.g. all samples are the digit "1"; in the ring data 1,000 samples landed in one region). - *Why:* D judges one sample at a time; G optimises against today's D; nothing rewards spreading out. - *Kinds:* complete, partial, within a mode, hopping. - *Detect:* sample grids, fixed-noise grids, class histograms. - *Fix:* minibatch std / minibatch discrimination. 2. **Instability / non-convergence:** "G fixes one weakness, D learns a new check, G switches trick, D switches back…": the players **chase each other in circles** instead of settling at 0.5. - *Symptoms:* oscillating losses, quality going backwards, one player winning outright, fragile settings. - *Causes:* gradient play circles the equilibrium; Adam momentum (β1 = 0.9) overshoots. - *Fixes:* Adam β1 = 0.5, a lower learning rate. 3. **D too strong:** D rejects every fake with total confidence: D(x) ≈ 1, D(G(z)) ≈ 0. - A perfect D is flat around the fakes, so **G's gradient vanishes** and G stops learning (vanishing gradient problem). "Balance matters." - *Fixes:* non-saturating G loss, one-sided label smoothing (real target 0.9), instance noise, WGAN-GP.

Rule: losses alone don't show quality; always look at the images. *Source: GAN-1 (slides 41, 55); GAN-2-3 (sections 03–05).*

Long answers (39–57)

39. Explain the process of thresholding and its types with examples. Definition: thresholding **converts a grayscale image into a binary (black-and-white) image by setting a cutoff value**: - pixel intensity **> threshold → white (255)**; - pixel intensity **≤ threshold → black (0)**.

It simplifies images into binary form for easier analysis (e.g. before finding contours).

Types: 1. **Global thresholding:** a single threshold for the whole image. `_, thresh_global = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)` 2. **Adaptive thresholding:** the threshold is calculated for **smaller regions** (different cutoffs for different regions), useful when **lighting/illumination varies**. `cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)` 3. **Otsu's method: automatically calculates the optimal threshold**, by **minimising intra-class variance**. `_, thresh_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)`

Examples and applications: - **document scanning/binarisation**: the Day 4 mini-project **Document Binarizer** produces a clean binary version of a scanned document using Otsu; - **medical imaging**: tumour segmentation; - **industrial inspection**: defect detection; - **object detection**: separating foreground from background.

The Day 4 lab compares global, adaptive and Otsu on the same image. *Source: W1-KC (Thresholding); D4.*

40. Discuss morphological operations (dilation, erosion, opening, closing) with diagrams and applications. Morphological operations focus on the **shape and structure** of objects, especially in **binary** images (they can also be applied to grayscale). They use a **structuring element**: a small binary matrix (rectangle, ellipse or cross) that probes the image. **Its size determines the effect**, e.g. `kernel = np.ones((5,5), np.uint8)`.

1. **Dilation: expands white regions**; connects broken parts, fills gaps in text characters. `cv2.dilate(img, kernel, iterations=1)`
2. **Erosion: shrinks white regions**; removes small noise, thins lines, removes stray dots. `cv2.erode(img, kernel, iterations=1)`
3. **Opening = erosion followed by dilation: removes small noise while keeping the main shapes intact** (e.g. cleaning scanned documents). `cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)`
4. **Closing = dilation followed by erosion: fills small holes inside objects** (e.g. gaps in letters or shapes). `cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)`

Diagram (■ = white/foreground):

```
Original      Dilation (thicker)   Erosion (thinner)
  ■■■             ■■■■■                  ·
  ■■■     →       ■■■■■          →       ■
  ■■■             ■■■■■                  ·

Opening:  ■■■  ·  (speck)   →  ■■■        tiny speck erased, shape kept
Closing:  ■ · ■  (gap)      →  ■■■        tiny gap filled, shape kept
```

Course analogy (marker and eraser): dilation = thicker strokes; erosion = thinner strokes; opening = erasing tiny specks; closing = filling tiny gaps.

Applications: - document processing (remove noise, enhance text readability); - medical imaging (highlight cells or tissues); - industrial inspection (detect defects); - object detection (clean up shapes before contour analysis).

Day 5 mini-project **Noise Removal in Scanned Documents:** opening, then closing, with a 3×3 kernel. *Source: W1-KC (Morphological Operations); D5.*

41. Explain Harris Corner Detection: method, pros, cons, and applications. - **Concept:** corners are **points where intensity changes in multiple directions** (where edges meet). They are **repeatable and uniquely localizable**, unlike flat patches or edges (sliding along an edge looks similar). - **Method:** uses **gradient information to compute a "corner response" score**. In the lab: `dst = cv2.cornerHarris(gray, 2, 3, 0.04)`, and the corners are marked as **red dots**. - **Pros:** simple and effective for detecting corners; good for **rigid structures** (buildings, checkerboards). - **Cons: not invariant to scale or rotation**; sensitive to image transformations. "Harris finds corners but misses scale/rotation changes." - **Applications:** image matching, tracking, **panorama stitching**, object recognition. - Other corner detectors named in the course: FAST, Shi-Tomasi.

⚠ *Your course files do not give the mathematical formula for the corner response. Source: W1-KC (Corner Detection); W2-KC (Harris); W2-Lab 1.*

42. Discuss the role of convolution, pooling, and activation functions in CNNs with examples. CNNs are neural networks specialised for images that **automatically learn spatial hierarchies of features**. Data flow: **Input → Convolution → Pooling → Flatten → Fully Connected → Output**.

1. **Convolution:**

2. **Role:** applies small filters (kernels) across the image to **detect local patterns**. Each filter produces a **feature map**: a 2D grid of how strongly it responds in each region.
3. **What it captures:** early layers → edges, corners, textures; deeper layers → shapes, objects, semantic features.
4. **Benefit: fewer parameters** than fully connected layers; preserves spatial locality.
5. **Stride** controls the step size; **padding** (usually zeros) prevents shrinking and lets filters cover edge pixels.
6. **Pooling:**
7. **Role: down-samples feature maps** by summarising regions. **Max pooling** keeps the strongest feature; **average pooling** smooths.
8. **Benefits:** reduces computation; gives **translation invariance**; controls overfitting.
9. **Activation functions:**
10. **Role:** introduce **non-linearity** so the network can learn complex patterns. Without them CNNs "would behave like linear models".
11. **ReLU:** fast, effective, widely used. **Sigmoid:** 0 to 1, for probabilities. **Tanh:** −1 to 1.

Example: the course's PyTorch MNIST CNN has two Conv2d layers → pooling and dropout → fully connected layers → digit class. It is trained with negative log-likelihood loss and Adam. Flatten bridges the feature maps to the fully connected layers. *Source: W2-KC (CNN Basics; Feature Maps; Pooling; Stride; Padding; Flatten).*

43. Explain the evolution of CNN architectures from LeNet to VGG, highlighting their key innovations. 1. **LeNet (1998):** - Yann LeCun; digit recognition (**MNIST**). - Architecture: **Convolution → Pooling → Fully Connected**. - **Contribution:** first proof that CNNs outperform traditional ML (like SVMs) for images; adopted in handwriting recognition and banking. *Proof of concept.* 2. **AlexNet (2012):** - **Won the ImageNet competition by a huge margin**; 5 conv + 3 fully connected layers. - **Innovations: ReLU** (faster training), **Dropout** (reduces overfitting), **GPU training** (massive speed-up). - Accuracy went from ~70% before to ~85%+ after. **Sparked the deep learning revolution in vision.** *Breakthrough in scale and performance.* 3. **VGG (2014, Visual Geometry Group):** - **Very deep (16–19 layers)** with **small 3×3 filters stacked together**. - **Contribution:** showed that **depth improves accuracy**; a standardised design widely used as a research baseline. *Depth and simplicity as design principles.*

Together they form the timeline of CNN evolution. Later, ResNet and EfficientNet solved the depth and efficiency problems of these deeper networks. *Source: W3-KC (CNN Architectures).*

44. Discuss the importance of ReLU and Dropout in training deep networks. ReLU: - **What it does:** keeps positive inputs and outputs zero for negative ones. - **Why it matters:** introduces **non-linearity** so networks learn complex patterns; **faster and simpler** than sigmoid or tanh; **helps reduce the vanishing gradient problem**. - **Analogy:** a filter that only lets positive signals through.

Dropout: - **What it does:** a **regularisation technique**; it randomly sets a fraction of neurons to zero during training. - **Why it matters:** forces the network to **learn robust features instead of memorising**; reduces reliance on specific neurons; **improves generalisation** to unseen data. - **Example:** 10 neurons, rate 0.3 → about 3 ignored each pass, so knowledge is distributed across neurons. - **Analogy:** a team exercise where some members sit out at random, so everyone learns to contribute.

Importance: "Together, ReLU and Dropout made CNNs like **AlexNet** train faster and generalise better, which is why they were revolutionary in 2012." The course's MNIST CNN also uses dropout for regularisation. *Source: W3-KC (ReLU and Dropout; AlexNet); W2-KC.*

45. Describe the four core challenges in computer vision (viewpoint, illumination, occlusion, scale) with examples. ⚠ *Your course files do not present these four as a single "core challenges" topic.* Each one appears in different places, as follows. 1. **Viewpoint:** - Features must be **repeatable**: detectable "even if the image changes (scale, rotation, lighting)". Perspective transformation "warps viewpoint". - *Examples:* **Haar** cascades work for **frontal faces** and are sensitive to **pose**; **Harris** is not invariant to rotation, while **SIFT/ORB** handle rotation. 2. **Illumination (lighting):** - Global thresholding uses one cutoff for the whole image, so **adaptive thresholding** is "useful for images with varying illumination". - **Optical flow** is sensitive to lighting changes; **Haar** is sensitive to lighting; **HOG** is robust to illumination changes; **SIFT** handles illumination. - *Course question:* RGB vs HSV for detecting traffic lights under varying lighting. HSV separates colour from brightness. 3. **Occlusion (object hidden):** - A listed challenge in **object tracking**. **KLT** "struggles with occlusion (object hidden)", optical flow is sensitive to occlusions, and Haar is sensitive to occlusion. - *Example:* tracking a moving ball that goes behind something. 4. **Scale:** - "**Scale changes**" are a tracking challenge. Harris is not scale-invariant, HOG is sensitive to scale changes, ORB is less robust for large scale changes, and **SIFT** is robust to scale. - **SSD** handles scale with multiple feature maps (a small and a large cat in one image).

Source: W1-KC; W2-KC; D1-3; D4; W3-KC; W1 question bank.

46. Describe the analogy of the Generator as a forger and the Discriminator as an inspector. The original GAN paper's analogy (Goodfellow et al., 2014): - The **counterfeiter** prints fake banknotes and tries to spend them without being caught: **the Generator**. - The **police** inspect notes and try to detect the fakes: **the Discriminator**. - "Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles."

The course's running example, the flower forger: - **Forger (G):** turns a blank idea (random noise z) into a fake flower. It **never sees a real flower** and learns **only from D's reaction**: the gradient tells each of G's weights "move this way and D's score for your image goes up". D is G's only teacher. - **Inspector (D):** looks at an image and gives a verdict P(real). It learns from real flowers what they look like. - **Rounds:** - round 1: G draws a random scribble, which D catches easily; - round 2: G learns "colourful blobs", and D starts checking shape; - round 3: G adds petals and a centre, and D checks petal detail; - **round N: G makes convincing flowers, and D can only guess, about 50/50.** - **Winners and losers:** if D catches the fake, G must update; if G fools D, D must update. In code both take one gradient step every batch: first D, then G.

Story	Maths	Code
forger's blank idea	z ~ N(0, I)	`torch.randn(batch_size, 100)`
forger paints	G(z)	`fake = generator(noise)`
inspector's verdict	D(x), D(G(z))	`discriminator(real)`, `discriminator(fake)`
fakes pass as real	p_g = p_data, D = ½	loss_d ≈ 1.386, loss_g ≈ 0.693

Source: GAN-1 (slides 13, 14, 15, 19, 24, 43).

47. Explain the architecture of the Generator in the notebook (layers, activations, output).

```
class Generator(nn.Module):
  def __init__(self):
    super().__init__()
    self.fc = nn.Sequential(
      nn.Linear(100, 256),
      nn.ReLU(),
      nn.Linear(256, 784),
      nn.Tanh())
  def forward(self, x):
    return self.fc(x).view(-1, 1, 28, 28)
```

- **Input:** a noise vector of **100** random numbers, `torch.randn(batch_size, 100)`.
- **Layers: 100 → 256 → 784**: noise vector → hidden layer (with **ReLU**) → one value per pixel.
- **Output activation: Tanh** squashes the outputs to **−1…1**, the same range as the real images after `Normalize((0.5,), (0.5,))`.
- **Output:** `.view(-1, 1, 28, 28)` reshapes the 784 numbers into a **1-channel 28×28 image**. A full batch has shape (64, 1, 28, 28).
- A small MLP with **100 inputs and 784 outputs**, trained with Adam (lr = 0.0002) and BCE loss, using `labels_real` as the target for its fakes.

Source: GAN-1 (slides 10, 25, 49, 52).

48. Explain the architecture of the Discriminator in the notebook (layers, activations, output).

```
class Discriminator(nn.Module):
  def __init__(self):
    super().__init__()
    self.fc = nn.Sequential(
      nn.Linear(784, 256),
      nn.LeakyReLU(0.2),
      nn.Linear(256, 1),
      nn.Sigmoid())
  def forward(self, x):
    return self.fc(x.view(-1, 784))
```

- **Input:** a 28×28 image, real or fake; `x.view(-1, 784)` **flattens** it.
- **Layers: 784 → 256 → 1**.
- **Hidden activation: LeakyReLU(0.2)** lets a small gradient through for negative inputs, **so G keeps getting a signal**.
- **Output: Sigmoid** turns the score into **one probability between 0 and 1**, P(real).
- **Mirror image of G:** G is 100 → 256 → 784; D is 784 → 256 → 1. A small MLP with 784 inputs and 1 output.
- Trained with `nn.BCELoss()` and Adam (lr = 0.0002): real → 1, fake → 0.

Source: GAN-1 (slides 10, 30, 49).

49. Compare RGB and HSV color models: advantages, limitations, and use cases in computer vision.

	RGB (BGR in OpenCV)	HSV
what it represents	mixes colours using **red, green and blue light** values	**Hue** (type of colour, an angle: 0° red, 120° green, 240° blue), **Saturation** (purity, 0% grey → 100% vivid), **Value** (brightness, 0% black → 100% brightest)
advantages	**great for screens**; the default image format (OpenCV loads BGR)	**closer to how humans perceive colour**; separates colour into intuitive parts (colour family, how colourful, how bright)
limitations	**not intuitive for humans**; brightness isn't separated from colour	⚠ *your course files list no limitations for HSV*
use cases	storing and displaying images (convert BGR→RGB for Matplotlib)	**colour detection, segmentation, image editing**: e.g. detecting **ripe fruit** by isolating the hue range for yellow or red "without worrying about brightness changes"

Conversion: `hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`. "Converting between colour spaces helps in tasks like colour detection." The Week 1 question bank applies this to **traffic-light detection under varying lighting** and **ripe-fruit detection**, where HSV's separation of brightness is the advantage. *Source: W1-KC (HSV; Why HSV is Useful); D1-3.*

50. Compare Haar cascades and HOG for face detection: methodology, strengths, weaknesses, and applications. Face detection = locating human faces in images or video. It is a fundamental step before face recognition, emotion analysis or biometrics.

	Haar cascades	HOG (Histogram of Oriented Gradients)
methodology	**Haar features = simple rectangular patterns** (like edge detectors); **AdaBoost** selects the most important features and combines them into a **cascade classifier**; run with `detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)`	captures **gradient-orientation distributions in local regions**: compute gradients → divide the image into **cells (e.g. 8×8)** → build orientation histograms → **normalise over blocks**
strengths	**fast and lightweight**; works well for **frontal faces in controlled environments**; real-time on a webcam	**robust to illumination changes**; captures **shape and edge information** effectively; lightweight and interpretable
weaknesses	**sensitive to lighting, pose and occlusion**; less robust in complex scenes; false positives and missed detections (tune `scaleFactor`, `minNeighbors`)	**computationally heavier** than Haar; may need more preprocessing; sensitive to scale changes
applications	face detector app (images/webcam), security systems, attendance monitoring, AR filters, face unlock	classical **face and pedestrian detection**, object classification

In short (course tip): Haar = **fast, lightweight**; HOG = **robust, heavier**. *Source: W2-KC (Face Detection); W2-Lab 6; W1-KC (HoG).*

51. Compare R-CNN, YOLO, and SSD architectures: processes, strengths, weaknesses, and ideal applications. *Object detection* = identifying **what** an object is and **where** it is.

	R-CNN (2014)	YOLO (2016)	SSD (2016)
full name	Region-based Convolutional Neural Network	You Only Look Once	Single Shot MultiBox Detector
process	1. generate **region proposals**; 2. **run a CNN on each region**; 3. classify and refine bounding boxes	**detection as a single regression problem**: boxes + class probabilities in **one forward pass**	uses **multiple feature maps at different scales**; detects small and large objects simultaneously
strengths	**high accuracy**	**real-time** (fast)	**faster than R-CNN, competitive accuracy**; handles multiple scales
weaknesses	**very slow**: thousands of regions per image	less accurate for **small objects**	slightly less accurate than R-CNN for very complex scenes
contribution	first major CNN-based detector	made detection practical in real time	**balanced speed and accuracy**
ideal for	**medical imaging** (tumours in MRI: accuracy > speed, offline)	**autonomous driving** (a pedestrian crossing: speed > accuracy)	**mobile vision tasks**; driving (multiple scales)
examples	a dog image → many proposed regions, each classified	many objects in a street scene in one shot	a small cat and a large cat in the same image

Key insight: the trade-off between accuracy and speed. Medical imaging → R-CNN (and successors like Faster R-CNN); autonomous driving → YOLO/SSD. *Source: W3-KC (Object Detection Architectures).*

52. Explain techniques used to overcome these challenges: data augmentation, histogram equalization, robust feature detection, and multi-scale feature extraction. ⚠ *Your course files do not present these four as one set of "challenge-overcoming techniques". This is what they say about each:* 1. **Data augmentation:** - The Day 2 mini-project is an **Image Augmentation Tool** that applies **translate, scale, rotate and perspective** transformations to an image. - GAN Class 1 lists data augmentation as a GAN use: "extra synthetic training images when real ones are rare or private". - ⚠ *The course does not discuss augmentation as a training technique beyond this.* 2. **Histogram equalization:** - Named as an enhancement technique based on histograms, which reveal under- or over-exposure. - ⚠ *No mechanism is given.* For uneven illumination the course uses **adaptive thresholding** ("useful for images with varying illumination"). 3. **Robust feature detection:** - Features should be **repeatable** "even if the image changes (scale, rotation, lighting)". - **SIFT** is robust to scale, rotation and illumination (Gaussian scale space, DoG extrema, orientation, 128-D descriptor). - **ORB** is fast and rotation invariant; **HOG** is robust to illumination. - Matching with **FLANN + Lowe's ratio test** and **RANSAC** (robust to outlier matches). 4. **Multi-scale feature extraction:** - **SSD's multiple feature maps** detect small and large objects together. - SIFT's **scale space**; Haar's `detectMultiScale`. - CNN layers go from edges (early) to objects (deep).

Source: D1-3; GAN-1; W1-KC; D4; W2-KC; W3-KC; W2-Lab 2, 6.

53. Discuss the training loop in detail: how labels_real and labels_fake are used, and how gradients flow in each step.

```
for epoch in range(epochs):                    # epochs = 2
  for real, _ in train_loader:
    batch_size = real.size(0)                 # last MNIST batch has only 32 images
    labels_real = torch.ones(batch_size, 1)
    labels_fake = torch.zeros(batch_size, 1)
    # Train Discriminator
    optimizer_d.zero_grad()
    outputs_real = discriminator(real)
    loss_real = criterion(outputs_real, labels_real)
    noise = torch.randn(batch_size, 100)
    fake = generator(noise)
    outputs_fake = discriminator(fake.detach())
    loss_fake = criterion(outputs_fake, labels_fake)
    loss_d = loss_real + loss_fake
    loss_d.backward(); optimizer_d.step()
    # Train Generator
    optimizer_g.zero_grad()
    outputs_fake = discriminator(fake)
    loss_g = criterion(outputs_fake, labels_real)
    loss_g.backward(); optimizer_g.step()
```

The labels (BCE: $-[y \cdot \log p + (1-y) \cdot \log(1-p)]$): - D's labels **come for free**: an image picked from the dataset gets **label 1** (`labels_real = torch.ones`), and one painted by G gets **label 0** (`labels_fake = torch.zeros`). The label means "**which pile did this come from**", not "is it pretty": a beautiful fake is still 0 when training D. - In **G's step**, the fakes get **`labels_real` (1)**. "Not cheating": the label here states **what G wants D to say**. "Loss is low when D is fooled." This is the **non-saturating loss**.

Gradient flow: - **D step:** `fake.detach()` **cuts the wire back to G**, and only `optimizer_d` steps, so the step changes **D's weights only**. - **G step:** the same fakes **without detach**, so the gradients **flow backwards through D into G**. D's weights are not updated here (only `optimizer_g` steps), so D acts as a **fixed judge**.

Setup: `criterion = nn.BCELoss()` ; two Adam optimizers with lr = 0.0002 (each only updates its own network); batch 64 (~938 batches per epoch). The printed losses are from the **last batch** of each epoch. **Worked BCE numbers:** D on a real image at 0.9 → 0.105, plus a fake at 0.2 → 0.223, total 0.328 (D doing well). G's loss on the same fake = $-\log 0.2 = 1.609$ (G must improve). *Source: GAN-1 (slides 28, 34–37, 49, 50, 52).*

54. Elaborate on contour analysis and object tracking: algorithms, challenges (occlusion, scale, clutter), and real-world applications. Contour analysis: - **Contours** are curves joining continuous points along the boundary of objects with the same intensity or colour. Contour analysis extracts and studies these boundaries, for **shape recognition, object detection and classification**. Key descriptors: **area, perimeter, centroid, bounding boxes, convex hulls**. - **Algorithm (Day 6):** 1. threshold to binary (`cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)`), or Canny edges (Day 3); 2. `cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)`; 3. draw contours; 4. **bounding box** (the smallest rectangle enclosing a contour, for localisation): `cv2.boundingRect`; 5. **convex hull** (the smallest convex polygon enclosing a contour, for shape simplification): `cv2.convexHull`; 6. **shape classification**: `cv2.approxPolyDP(cnt, 0.02*cv2.arcLength(cnt, True), True)`, then count vertices: 3 = triangle, 4 = rectangle, more than 6 = circle, otherwise polygon.

Object tracking: - **Definition:** continuously locating (following) a moving target across consecutive video frames. - **Approaches:** - detection-based (detect in each frame with YOLO / Faster R-CNN and link the detections); - tracking-by-motion (optical flow or correlation filters); - hybrid (SORT, DeepSORT). - **Optical flow:** estimates pixel motion between frames, assuming a pixel's brightness stays constant. **Dense** = every pixel (Farneback; motion visualised with HSV colour coding); **sparse** = selected keypoints. Methods: Lucas–Kanade (local), Horn–Schunck (global smoothness). - **KLT tracker:** 1. detect "good features to track" (corners): `cv2.goodFeaturesToTrack`; 2. follow them with Lucas–Kanade: `cv2.calcOpticalFlowPyrLK`. Tune `maxCorners` and `winSize`. - **Kalman filter:** predict the next position from a motion model, then correct it with the measurement. It handles noisy data and smooths trajectories; best for linear motion with Gaussian noise.

Challenges: - **occlusion:** KLT struggles when the object is hidden; optical flow is sensitive to occlusions; - **scale changes**; - **cluttered backgrounds** and **multiple objects**; - large motion jumps; lighting changes; - objects changing appearance; - KLT needs re-initialisation when features are lost.

Applications: - contours: object detection and tracking, industrial inspection, medical imaging (cell shape analysis), shape detector; - tracking: surveillance and CCTV, sports analytics and replay analysis, autonomous driving and driver assistance, human–computer interaction, traffic flow visualisation, unusual-movement detection.

Source: W1-KC (Contours; Object Tracking; Optical Flow; Kalman Filters); D6; W2-KC (Object Tracking Basics); W2-Lab 3.

55. Elaborate on the Bag of Visual Words pipeline: feature extraction, clustering, histogram representation, and classification. Idea: inspired by text analysis, where documents are represented as **word-frequency histograms**. In CV, local image features are clustered into **"visual words"**. BoVW **converts variable-length feature sets into fixed-length histograms**, so standard ML algorithms can classify them. Analogy: words ↔ visual words, document ↔ image, word frequency ↔ histogram. 1. **Feature extraction:** extract **local features** from the images (e.g. **SIFT** or ORB descriptors). 2. **Feature clustering:** use clustering (commonly **k-means**) to group the descriptors; **each cluster centre is a visual word**. Like grouping synonyms in language, it groups similar visual patterns. Lab: `KMeans(n_clusters=k, random_state=42)`. 3. **Histogram representation: assign each feature to its nearest cluster (visual word) → count the occurrences** of each word → the histogram becomes the image's representation. **Benefit:** a **fixed-length vector** regardless of how many features were extracted. 4. **Classification:** feed the histograms to classifiers like **SVM or kNN**. Mini-project **Texture Classifier**: an SVM trained on BoVW histograms.

Number of clusters k: it sets the vocabulary size and affects classification accuracy. **Applications:** texture classification, scene recognition, object categorisation, material recognition, surface inspection, texture-based retrieval, fabrics in fashion, natural textures in environmental monitoring. *Source: W2-KC (Bag of Visual Words); W2-Lab 5.*

56. Elaborate on how modern architectures (ResNet, EfficientNet, ViT) solved depth and efficiency challenges, and analyze their impact on real-world vision tasks. The problem: LeNet, AlexNet and VGG showed the power of CNNs, but **deeper networks faced vanishing gradients and computational inefficiency**. Modern architectures addressed **depth, efficiency and scalability**. 1. **ResNet (2015): depth.** - Very deep networks had **vanishing gradients** (weights stopped updating). - **Residual (skip) connections**: the network **learns residuals instead of direct mappings**, and gradients flow more easily. - Networks of **50, 101 and 152 layers** became trainable; state-of-the-art on ImageNet. - **Impact:** medical imaging. 2. **EfficientNet (2019): efficiency.** - Scaling deeper, wider or to higher resolution was inefficient and unbalanced. - **Compound scaling**: depth, width and resolution scaled systematically with a single formula. - High accuracy with **fewer parameters**: it outperforms larger CNNs while being lightweight. - **Impact: mobile vision apps**. 3. **Vision Transformer (ViT): global context.** - CNNs struggle with **long-range dependencies**. - ViT splits the image into **16×16 patches** treated as tokens; a **transformer encoder with attention** learns relationships across patches; a classification token summarises the image. - **Benefits:** captures global context better than CNNs; **scales well with large datasets**; flexible (classification, detection, segmentation); inspired hybrid CNN + Transformer models. - **Impact:** image classification (ImageNet), medical imaging (global patterns), video understanding, multimodal (vision + language). It **shifted computer vision from CNN-centric to transformer-centric approaches**.

Analysis: ResNet made very deep networks trainable, and EfficientNet made scaling efficient and practical. Together they represent the modern era of CNNs, and their ideas influenced later models and even ViT. ViT then added global context. *Source: W3-KC (Why Study Modern CNNs?; Vision Transformers & Attention).*

57. Analyze the mathematical foundation of GANs: minimax game, optimal Discriminator, Nash equilibrium, and why the non-saturating generator loss is used. 1. Minimax game:

min_G max_D V(D, G) = E_x[log D(x)] + E_z[log(1 − D(G(z)))]

- D **pushes V up**: it says "real" to real data (D(x) → 1) and pushes D(G(z)) → 0.
- G **pushes V down**: it pushes D(G(z)) → 1, and can only affect the second term.
- **Zero-sum game:** the payoffs always add to zero (D gets V, G gets −V), so D's gain is G's loss. Minimax = "G minimises what the best D can maximise".
- **Generative goal:** move the generator's distribution **p_g** until it sits on top of the data distribution **p_data**.

2. Optimal Discriminator (for a fixed G):

D*(x) = p_data(x) / (p_data(x) + p_g(x))

i.e. "how much of the probability at x comes from real data?"

3. Nash equilibrium: - **Definition:** a choice for every player such that **no player can do better by changing only their own choice**. "No regrets" test: knowing what the others did, would you change your move? - **GAN's equilibrium:** against D*, G does best when **p_g = p_data**. Then **D*(x) = ½ everywhere** and **V = −log 4 ≈ −1.386**. D can do no better than a coin flip, so neither player can improve alone. - Notebook reference values: loss_d = 2 ln 2 ≈ 1.386, loss_g = ln 2 ≈ 0.693. - **In practice:** "the theory guarantees it exists, not that gradient training will reach it". In the x·y game, gradient play **circles** the equilibrium: simultaneous updates spiral outward, alternating updates orbit for ever. A smaller lr only slows the circling. A GAN searches for a **saddle point**, not a valley.

4. Why the non-saturating generator loss: - **Early in training** fakes are bad, so **D(G(z)) ≈ 0**. - The original term **log(1 − D(G(z))) is almost flat there → tiny gradient → G barely learns**. - The notebook's term **−log D(G(z)) is steep there → a strong gradient exactly when G needs it**. - Same goal (D(G(z)) → 1). In code: `criterion(outputs_fake, labels_real)`. (Also suggested in Goodfellow et al. 2014, Section 3.) - It softens the "D too strong" vanishing-gradient problem, but "a D that is far ahead can still stall G".

Source: GAN-1 (slides 9, 38, 39, 40); GAN-2-3 (sections 01–03).